High Performance Programming

Lecture 2:

”Parallel Processing: Communications and Data Structures”

Lecturer: Ramoni Adeogun

AALBORG
UNIVERSITET

Content

1. Network models
2. Communication in parallel systems
3. Data Structures* (Self study)
4. Exercise

1. Network models

Network models

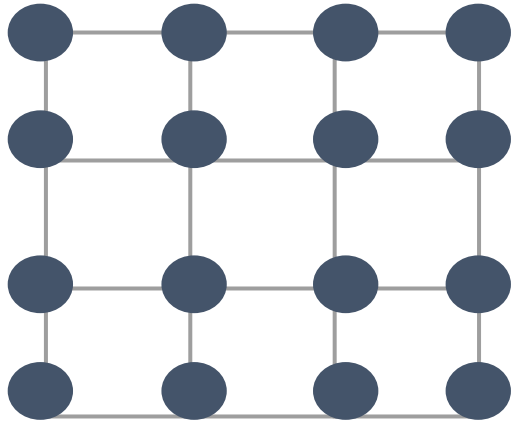
- Several processors are interconnected using physical links and the following assumptions are made:
  - Each has its own local memory
  - No common memory that can be accessed by all processors
  - Connected directly with physical links and the interconnection topology is called the **network topology**
  - If two processors are adjacent, data can be moved from one processor to the other directly
  - In one clock cycle, a unit operation takes place in any processor
  - A processor can communicate a data to any of its adjacent processor in a unit time.

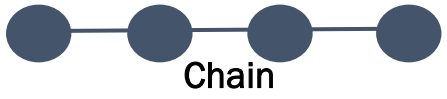
Specification for designing a network model for a problem:

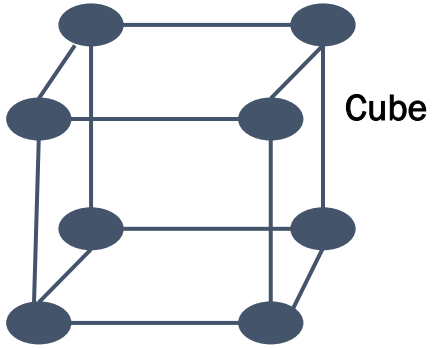
Network Topology	Investigate the problem and select a interconnection topology
Input Configuration	Distribution of input among processors
Output Configuration	Distribution of output in various processors

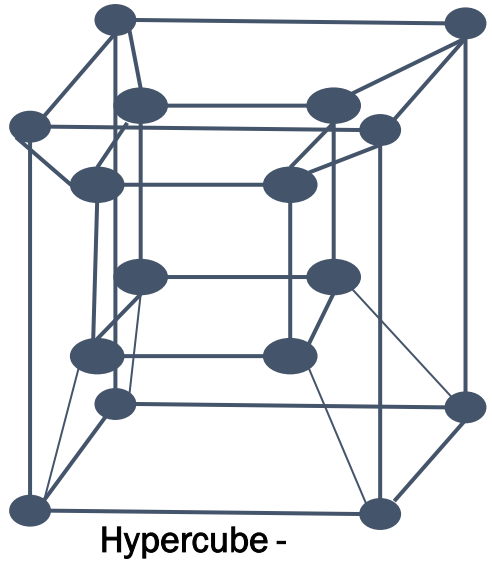
Network Model

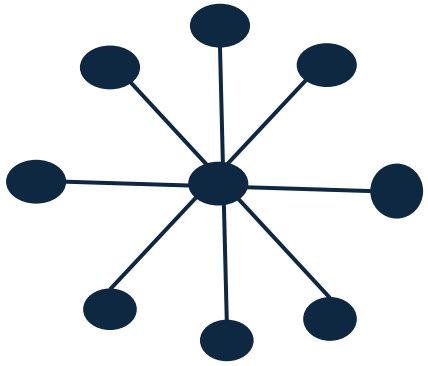
- Interconnection network represented by a graph, $G = (V, E)$
- Each $v \in V$ represents a processor
- An edge $\{u, v\} \in E$ represents a two-way communication link between processors u and v
- Network is asynchronous
- All coordination/communication must be done by explicit message passing
- No global shared memory

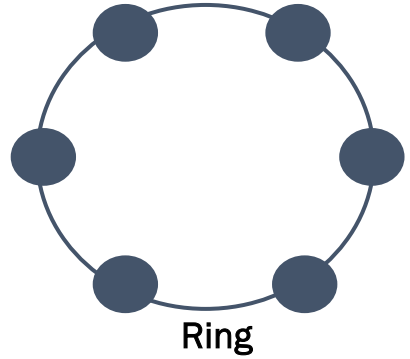






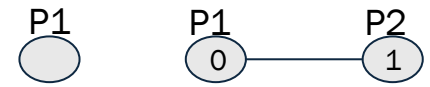






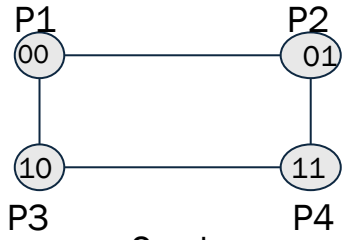
Hypercube (d-cubes)

- A d-cube is a d-regular graph with 2^d nodes
  - Each node in a d-regular graph has degree d.
  - Distance between two nodes is the number of different bits.
  - Communication between adjacent nodes takes 1 unit time.
- In a hypercube with $n = 2^d$ nodes
  - communication between any 2 nodes take at most $\log_2(n)$ time.
  - How does this compare with a complete graph?

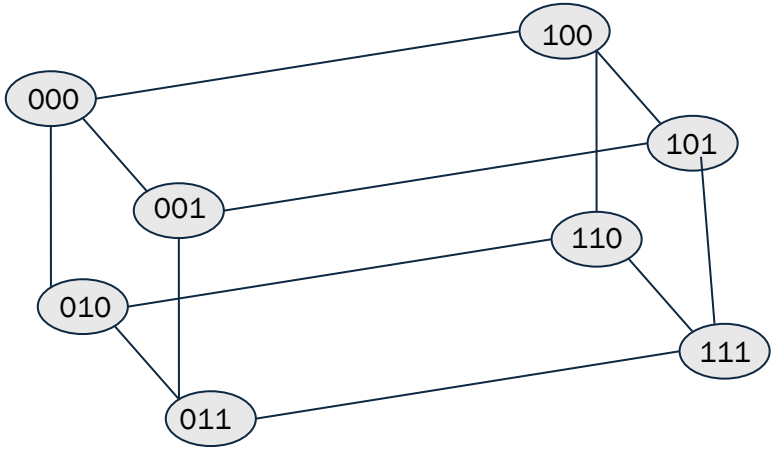


0-cube

1-cube



2-cube



3-cube

Example: Sum of $n = 2^d$ numbers on a d-dimensional hypercube

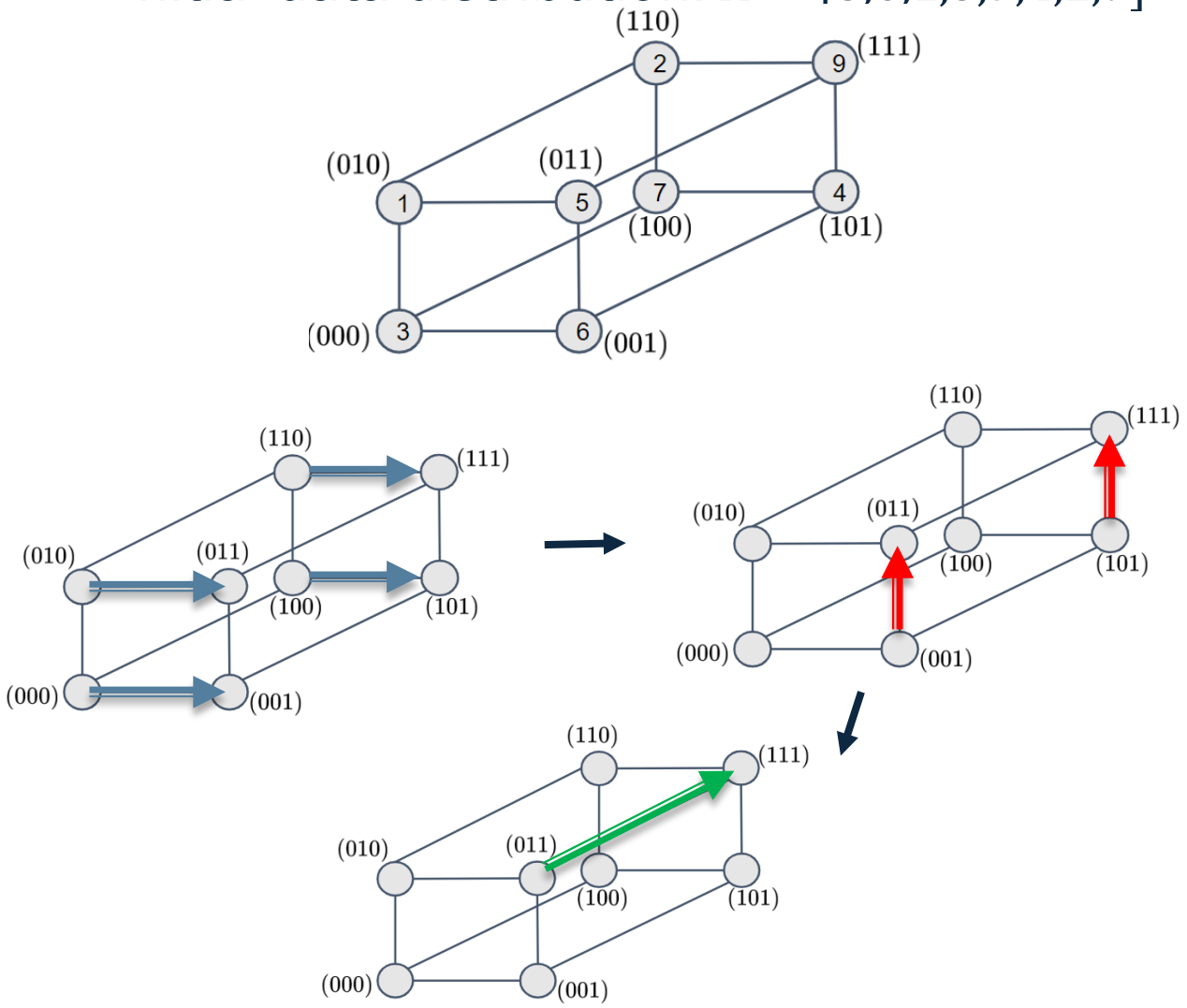
- Assignment(Input configuration): A_i is on processor P_i
- Computation: $S = \sum_i A_i$

Process:

- **Step1**: Processors in sub-cube XX0 send their data to corresponding processors in sub-cube XX1.
- **Step 2**: Processors in sub-cube X01 send their data to corresponding processors in sub-cube X11.
- **Step 3**: Processor 011 send its data to processor 111

Parallel reduction

- Initial data distribution: $A = [3,6,1,5,7,4,2,9]$



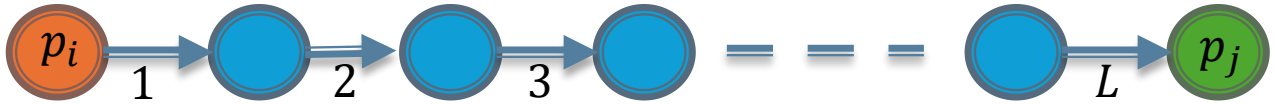
2. Communication

Basic Communication

- Many interactions in practical parallel programs occur in **well-defined patterns** involving groups of processors.
- Efficient implementations of these operations can improve **performance, reduce development effort and cost, and improve software quality**.
- Efficient implementations must leverage **underlying architecture**. For this reason, we refer to specific architectures here.
- Group communication operations are built using **point-to-point messaging primitives**.
- We assume that the network is **bidirectional**, and that **communication is single-ported**.

Communication cost

How long does message exchange between two processors on a network model take?



Communication cost:

$$T_{com} = t_s + t_w \times M \times L$$

- t_s → Message preparation time (latency per message)
- t_w → Unit transfer time
- M → Message length
- L → Number of traversed

Bound on communication cost on network models

- Bound on communication cost → maximum communication cost possible
  - Communication cost with L as the maximum distance between any two nodes on the network

- **Chain**

$$T^*_{com} = t_s + t_w M(p-1)$$

- Ring

$$T^*_{com} = t_s + t_w M \left\lfloor \frac{p}{2} \right\rfloor$$

- Mesh

$$T^*_{com} = t_s + 2t_w M(\sqrt{p}-1)$$

- Hypercube

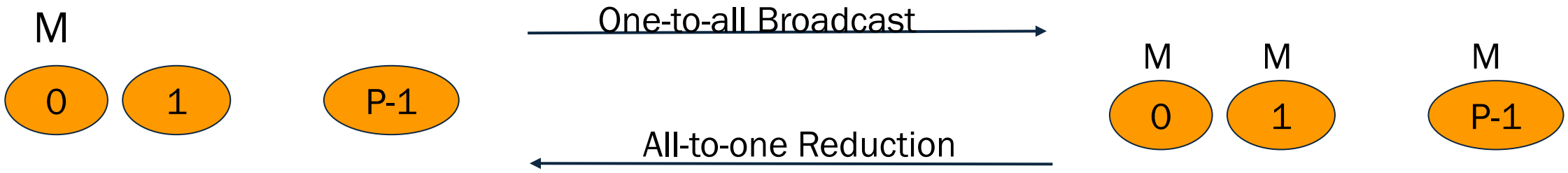
$$T^*_{com} = t_s + t_w M \log_2(p)$$

One-to-All Broadcast and All-to-One Reduction

- In **one-to-all broadcast**
  - One processor has a piece of data (of size M) it needs to send to everyone.

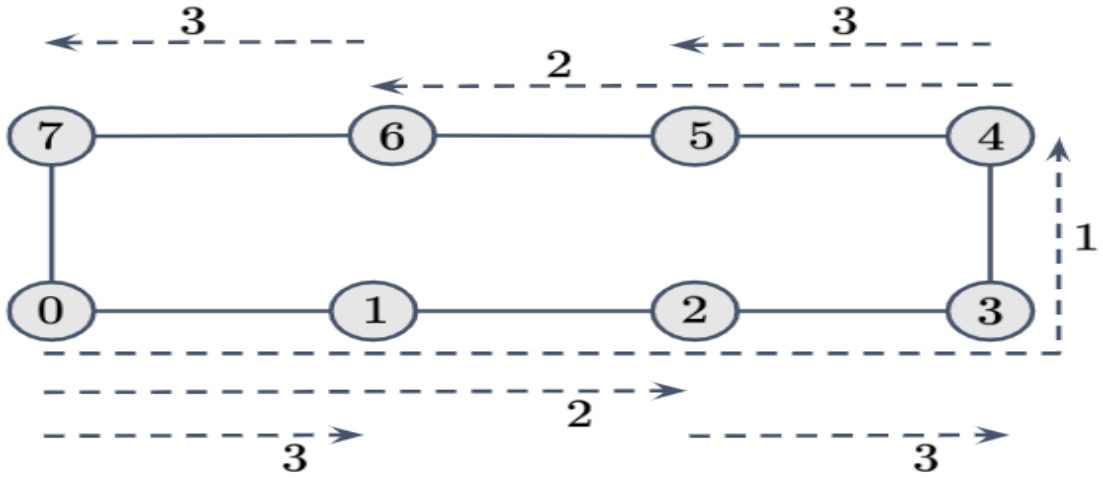
- The dual of one-to-all broadcast is all-to-one reduction.

- In **all-to-one reduction**, each processor has M units of data.
  - These data items must be **combined piece-wise**
    - using some **associative operator**, such as addition or min
    - and the result made available at **a target processor**.

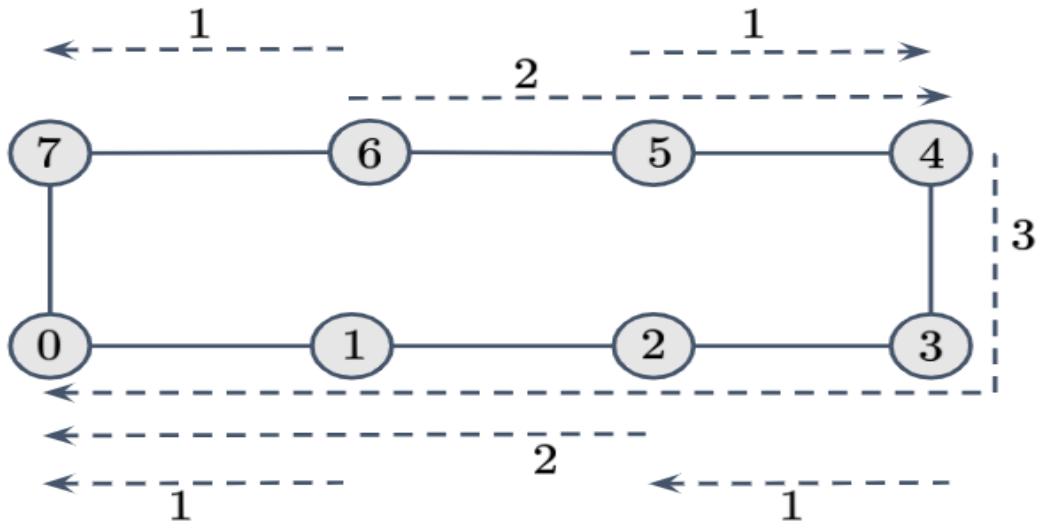


One-to-All Broadcast and All-to-One Reduction on Rings

- Simplest way is to send $p - 1$ messages from the source to the other $p - 1$ processors - this is not very efficient!!.
- Use recursive doubling → source sends a message to a selected processor.
- We now have two independent problems defined over halves of the network.
- Note: Reduction can be performed in an identical fashion by inverting the process.



One-to-all broadcast



All-to-one reduction

Example: One-to-All message broadcast on a ring

- **Naive approach**
  - Source sequentially send message to other $p-1$ processors
    - Inefficient
    - Underutilization of resources

- **Recursive doubling** → more efficient approach
  - Complete in $\log_2(p)$ steps

- What is the communication cost with p processors?

Communication cost:

- Naive approach:

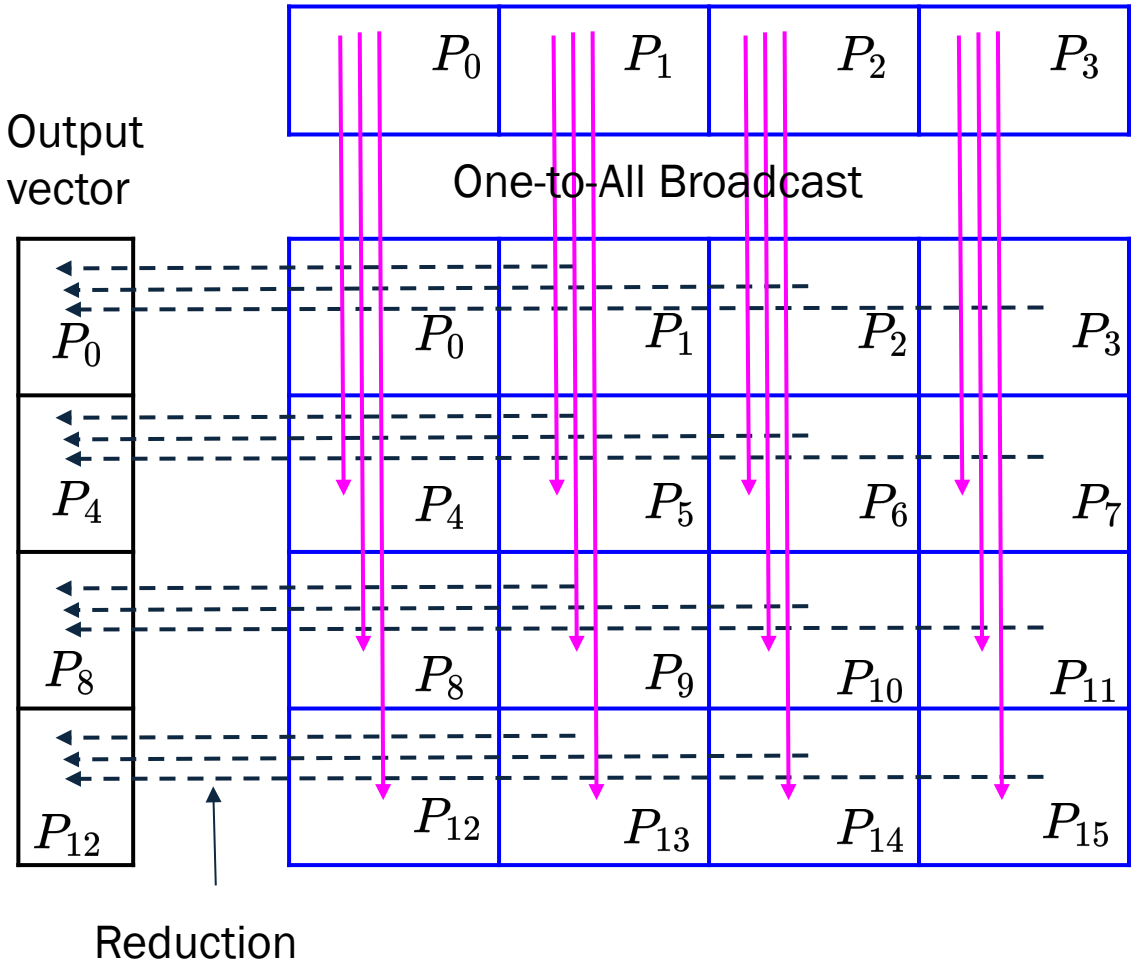
$$T_{com} = (p-1)(t_s + t_w M)$$

- Recursive doubling

$$T_{com} = \log_2(p)\,(t_s + t_w M)$$

Broadcast and Reduction: Example

- Consider the problem of **multiplying a matrix with a vector.**
  - The **n x n** matrix is assigned to an **n x n** (virtual) processor grid.
  - The vector is assumed to be on the **first row of processors**.
- The first step of the product requires a **one-to-all broadcast of the vector element along the corresponding column of processors**. This can be done concurrently for all n columns.
- The processors compute local product of the vector element and the local matrix entry.
- In the final step,
  - the results of these products are accumulated to the first row using **n concurrent all-to-one reduction operations along the rows** (with the sum operation).



One-to-All Broadcast and Reduction on a Mesh

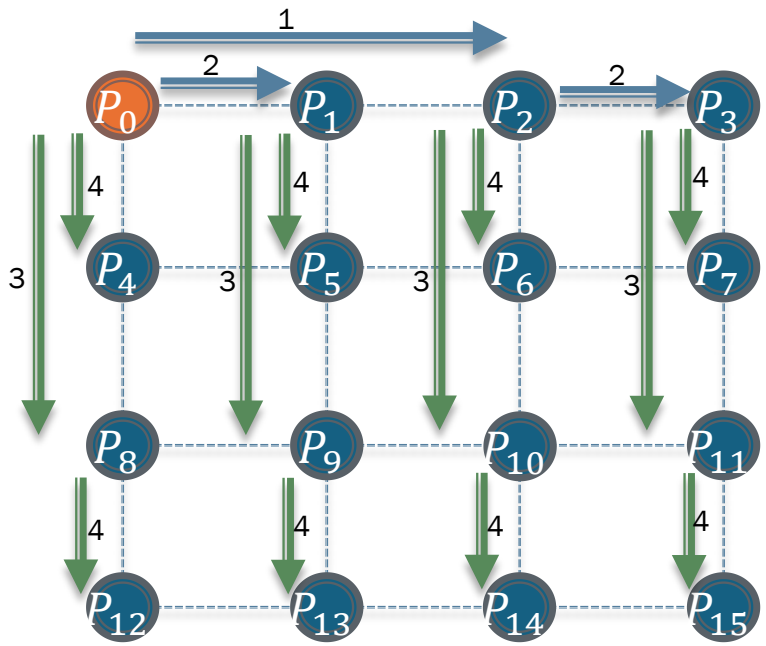
Broadcast and reduction on a mesh

We can view each row and column of a square mesh of p nodes as a linear array of $\sqrt{p}$ nodes.

Broadcast and reduction operations can be performed in two steps -

- Step 1: One-to-All along a row
- Step 2: One-to-All along each column concurrently.

Note: This process generalizes to higher dimensions as well.



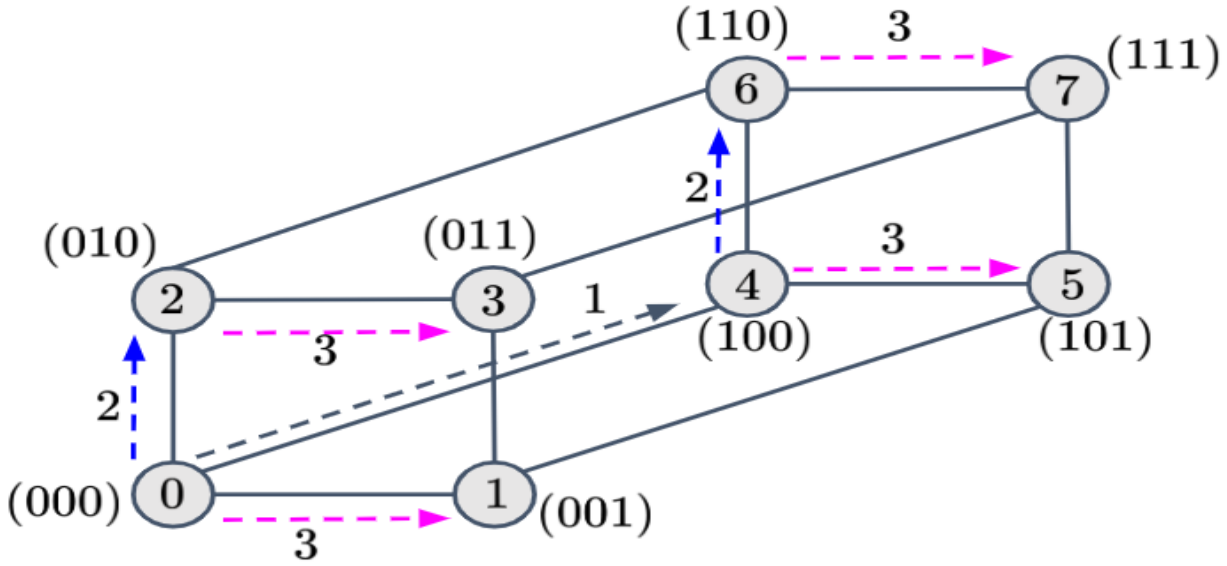
What is the communication cost?

$$T_{com} = \log_2(p)(t_s + t_w M)$$

Broadcast and Reduction on a Hypercube

A hypercube with 2^d nodes can be regarded as a d-dimensional mesh with two nodes in each dimension.

The mesh algorithm can be generalized to a hypercube and the operation is carried out in $d\ (=\ \log_2(p))$ steps.

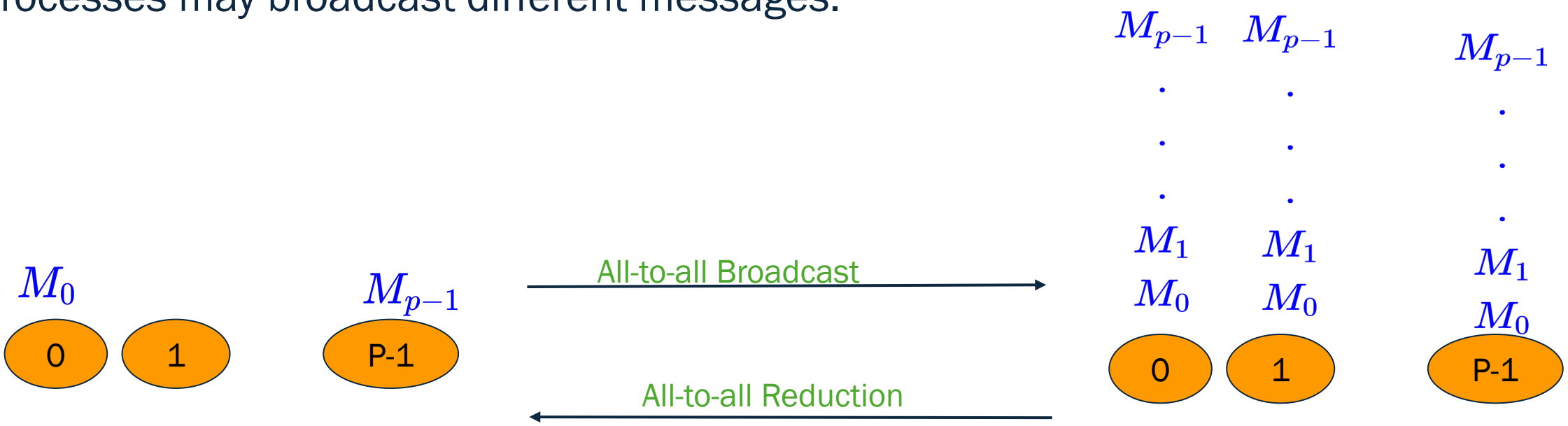


One-to-all broadcast on a three-dimensional hypercube

All-to-All Broadcast and Reduction

- Generalization of broadcast in which each processor is the source as well as destination.
- Applications: Parallel algorithms requiring global sharing of information, such as collective communication in distributed systems.

A process sends the **same m-word message to every other process**, but different processes may broadcast different messages.



All-to-All Broadcast on a Ring

Simplest approach:

- Use p individual one-to-all broadcasts → Each process broadcasts its data to all other processes.

Algorithm: All-to-All Broadcast

1. Initialization:
   - Each process p_i holds its own local data M_i
2. Steps:
   - For $i = 0$ to $p - 1$ iterations
     - Process p_i broadcasts M_i to all other processes.

Communication cost:

$$T_{comm} = p(p-1)(t_s + t_w M)$$

All-to-All Broadcst on a Ring

Ring Topology

- Each process is connected to two neighbors (left and right).
- Communication occurs sequentially, passing messages around the ring.

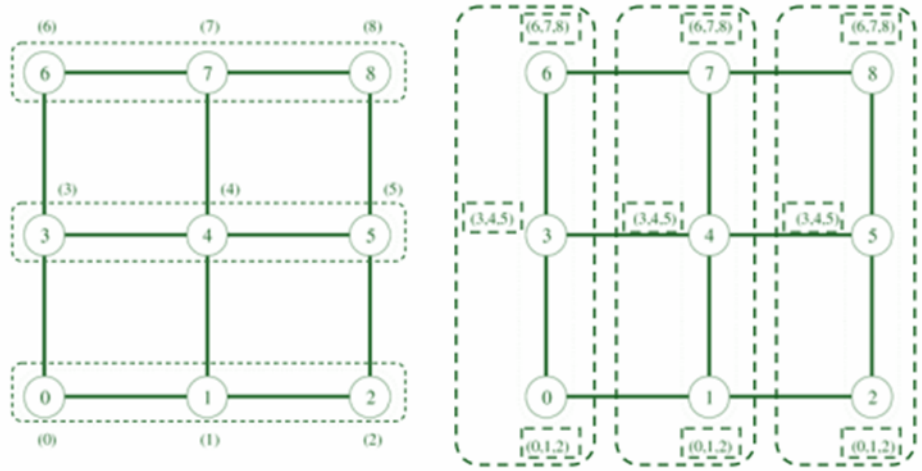
Algorithm: All-to-All Broadcast

1. Initialization:
   - Each process p_i holds its own local data M_i
2. Steps:
   - For p-1 iterations
     1. Send the current block to the right neighbor.
     2. Receive a block from the left neighbor
     3. Store the received block

Communication cost

$$T_{comm} = (p-1) \times (t_s + t_w M)$$

All-to-All Broadcast on a Mesh



- Mesh topology
  - Most processors has 4 neighbors (up, down, left, right).
  - Boundary processors have fewer neighbors.

Algorithm: All-to-All Broadcast

1. Initialization
   - Each process p_i holds its own data block M_i.
2. Steps
   - **Phase 1**: Perform a series of row-wise broadcasts (or column-wise), where each row (or column) of the mesh broadcasts data to all processes in the row (or column).
   - **Phase 2**: Perform a column-wise (or row-wise) broadcast to ensure that each process gets data from every other process.

<u>Communication cost:</u>

Phase 1: $\sqrt{p}$ simultaneous all-to-all broadcasts → $T_1 = (t_s + t_w M)\left(\sqrt{p} - 1\right)$

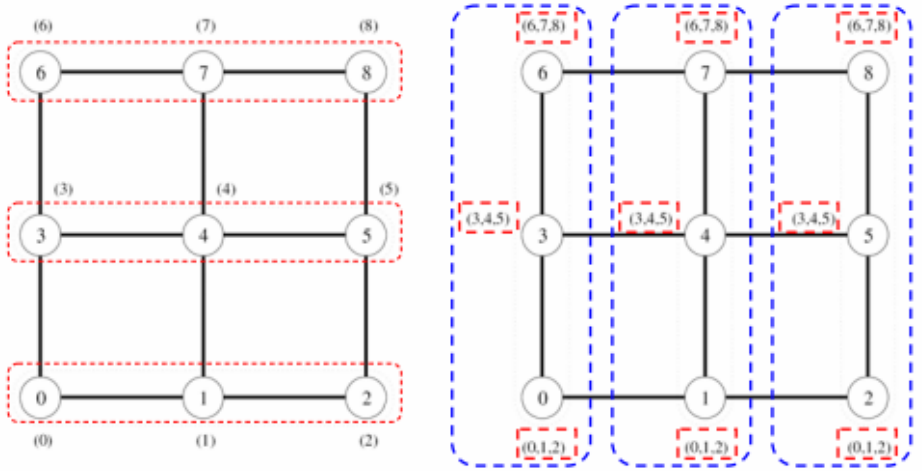
Phase 2: $\sqrt{p}$ simultaneous all-to-all broadcasts → $T_2 = \left(t_s + t_w \sqrt{p} M\right)\left(\sqrt{p} - 1\right)$

$$T_{comm} = 2t_s\left(\sqrt{p} - 1\right) + t_w M(p - 1)$$

Communication Cost Analysis

On a ring, the time is given by:

$$T_{com} = (t_s + t_w \times M)(p-1)$$



On a mesh, the time is given by:

$$T_{com} = 2t_s(\sqrt{p}-1) + t_w \times M(p-1)$$

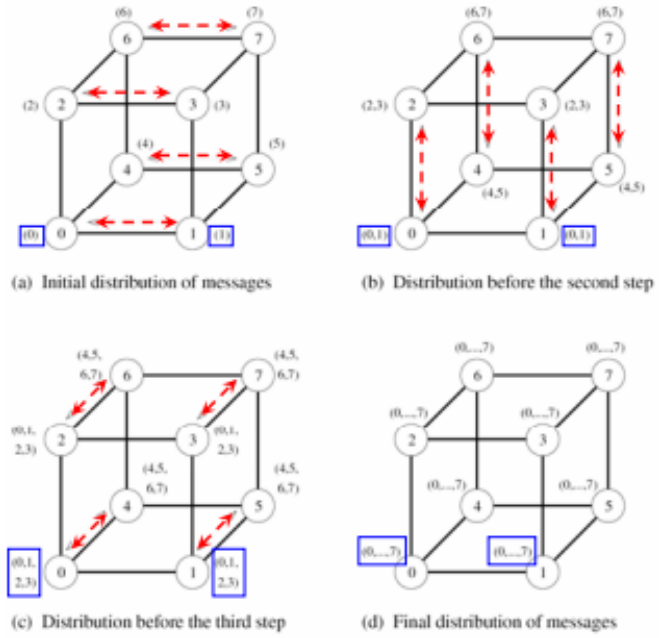
On a hypercube, we have:

log(p) steps

Message of size: 2^{i-1}

$$T_{com} = \sum_{i=1}^{\log_2(p)} \left(t_s + 2^{i-1} t_w M\right)$$

$$= t_s \log_2(p) + t_w M(p-1)$$

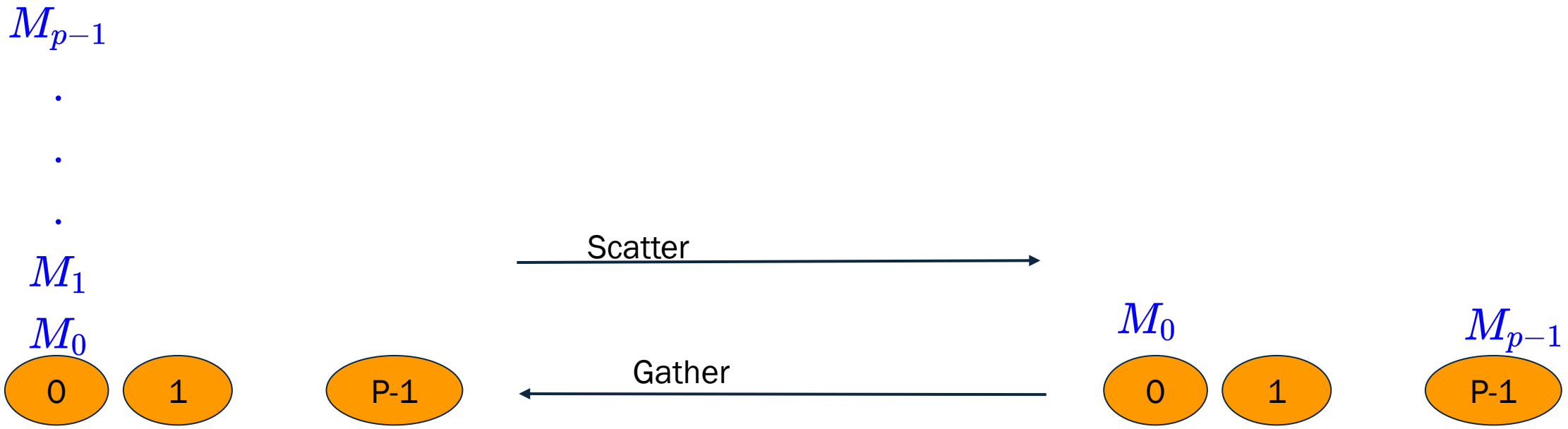


(a) Initial distribution of messages (b) Distribution before the second step

(c) Distribution before the third step (d) Final distribution of messages

Scatter and Gather

- Scatter - a single node send a unique message to every other node
  - Also called **one-to-all personalized communication**
- Gather (or concatenation) - a single node collects a unique message from every other node.

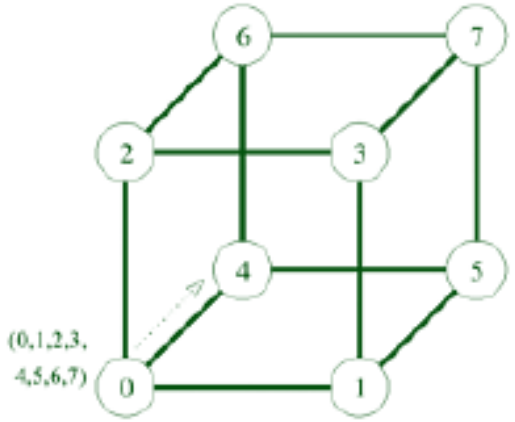


Scatter Operation on a 3-cube

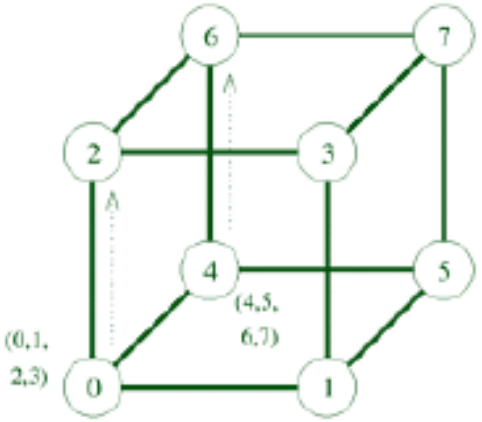
- Similar to the one-to-all broadcast
- Except for message content difference

Communication cost:

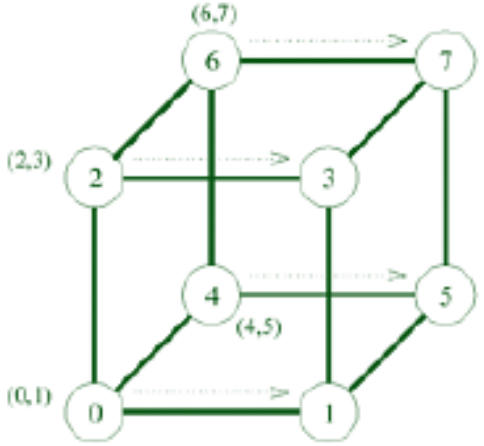
$$T_{com} = t_s \log_2 (p) + t_w M(p-1)$$



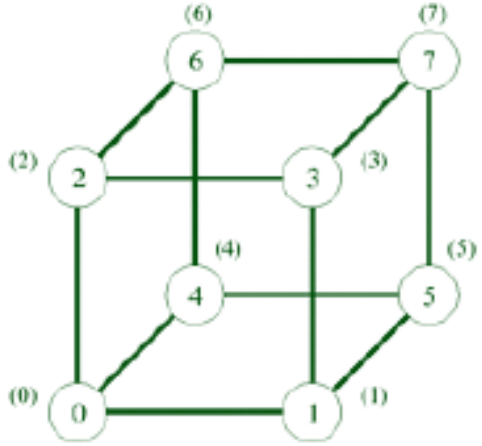
(a) Initial distribution of messages



(b) Distribution before the second step



(c) Distribution before the third step



(d) Final distribution of messages

3. Data structures

Role of Data structures in HPC

- Data structures directly influence computational complexity and memory access patterns.

- Key considerations:
  - Memory locality (e.g., cache efficiency).
  - Minimizing access overhead.
  - Optimizing for parallel read/write access.

Common data structures in parallel processing

- **Arrays**
  - Simple, cache-friendly, and suited for vectorization.
- **Linked Lists**
  - Suitable for dynamic memory usage but less cache-efficient.
- **Queues**
  - Useful for task scheduling and load balancing
- **Hash Tables**
  - Efficient lookups but challenging for parallel access.
- **Trees (e.g., Binary Trees, AVL Trees)**
  - Support hierarchical data but may require careful balancing for parallel updates
- **Graphs**
  - Model complex relationships such as social networks, road networks and dependencies in computation.

Arrays

- Fundamental for numeric computation and vectorized operations.
- Contiguous memory layout ensures efficient use of caches.

- Advantages
  - Fast random access ($O(1)$).
  - Easy to parallelize for tasks such as summation, sorting, and matrix operations.
  - Compatible with SIMD and GPU programming.

- Challenges:
  - Static - resizing requires creating new arrays → Limited flexibility for dynamic data structures.

- Example Usage:
  - Image processing (2D arrays for pixel grids).
  - Scientific computing (numerical simulations, linear algebra).

Linked List

- Dynamic memory allocation supports insertion/deletion without resizing.
- Useful in memory-constrained scenarios.

- Advantages:
  - Easily grows or shrinks dynamically.
  - Efficient for sequential access.

- Challenges:
  - Poor cache performance due to non-contiguous memory.
  - Harder to parallelize: Dynamic pointers introduce overhead.

- Example Usage:
  - Task scheduling (e.g., work-stealing queues).
  - Memory management in dynamic allocation.

Queues

- Used for task scheduling and load balancing in parallel systems.
- First-in, first-out (FIFO) structure aligns with many workflows.

- Advantages
  - Simple and intuitive for producer-consumer models.
  - Work-stealing queues enhance dynamic load balancing.

- Challenges
  - Contention arises when multiple threads access the same queue.
  - Managing concurrent access without introducing bottlenecks.

- Example Usage
  - Thread pools in multithreaded programming.
  - Load balancing in distributed computing

Hash Tables

- Enable efficient lookups, insertions, and deletions ($O(1)$ average time).
- Widely used for key-value mapping in parallel applications.

- Advantages
  - Highly efficient for unordered data.
  - Can scale with distributed hash table designs (e.g., in distributed databases).

- Challenges:
  - Collisions: Require strategies like chaining or open addressing.
  - Parallel modifications require synchronization or partitioning.

- Example Applications:
  - Caching systems (e.g., DNS lookup).
  - Distributed storage indexing.

Trees

- Ideal for hierarchical data representation and search operations.

- Advantages
  - Balanced trees (e.g., AVL, Red-Black) ensure logarithmic depth for operations ($O(\log n)$).
  - Easy to split for parallel traversal or updates.

- Challenges:
  - Balancing overhead: Requires rebalancing after insertions/deletions.
  - Parallel modifications are hard to manage without locking.

- Example Usage
  - Search trees in databases (e.g., B-trees).
  - Computational geometry (e.g., KD-trees, Quad-trees).

Graphs

- Useful for modeling complex relationships, e.g., social networks, road networks, dependencies in computation.
- Key to solving problems like shortest paths, spanning trees, and network flows.

- Graph Representations:
  - Adjacency Matrix:
    - Fast lookups ($O(1)$) for edge existence.
    - High memory cost ($O(V^2)$).
  - Adjacency List:
    - Space-efficient ($O(V + E)$).
    - Slower lookups ($O(V)$).
- Challenges in Parallelizing Graph Algorithms
  - Irregular data access patterns.
  - Dynamically changing workloads

- Applications in Parallel Systems
  - Breadth-First Search (BFS) → parallelize by processing frontier nodes simultaneously.
  - Minimum Spanning Tree → divide the graph into subgraphs and merge results.

Summary

- Communication in parallel algorithms can lead to significant overhead
  - Cost depends on the network model and type of communication, e.g., broadcast or reduction
  - Design of parallel algorithm must take communication costs into consideration

- To determine communication cost, we need to account for
  - Message preparation time/latency
  - Transfer time per message on each link
  - Number of traversed links by the message

- Data structures are crucial for parallel algorithm design
  - Structures for sequential algorithms can be used - lists, array, graphs and trees
  - Pay attention to the benefits and challenges associated with each data structure